

استاد: دکتر داوودآبادی

نام درس: مبانی پردازش زبان و گفتار

گزارش: HW4

نام و نام‌خانوادگی: نیوشا یقینی

شماره دانشجویی: ۹۸۵۲۲۳۴۶

اردیبهشت ۱۴۰۴

فهرست مطالب

1. گام اول: تولید داده مصنوعی در حوزه پزشکی ۳
2. گام دوم: تنظیم مدل روی دیتاست ساخته شده ۱٤
3. گام سوم: ارزیابی عملکرد مدل جدید ۲٤

۱. گام اول: تولید داده مصنوعی در حوزه پزشکی

با استفاده از تکنیک‌های پرامپت‌سازی (prompt engineering) و یا استفاده ازChatGPT ، یک مجموعه داده مصنوعی تولید کنید. برای تولید این دیتاست از منبع داده شده استفاده کنید.

- **فرمت داده‌ها باید به صورت زیر باشد:**
  - **ورودی: علائم بیمار (symptoms)**
  - **خروجی: تشخیص بیماری (diagnosis)**

نمونه :

```
[
  {
  "symptom": " مردی ۵۰ ساله با کاهش ناگهانی بینایی در چشم راست و بدون درد، با نوار چشم‌سنجی حاکی از نبود جریان خون شبکیه مراجعه کرده است.",
  "diagnosis": "انسداد شریان مرکزی شبکیه"
  }
]
```

- **تعداد نمونه‌های تولیدی :۱۰۰ جفت پرسش و پاسخ**
- **دیتاست به زبان فارسی و فرمت آن json باشد.**

توضیحات اولیه:

تکنیک‌های پرامپت‌سازی (Prompt Engineering) به روش‌هایی گفته می‌شود که با استفاده از آن‌ها می‌توان خروجی مدل‌های زبانی مانند ChatGPT را به‌صورت دقیق‌تر، هدفمندتر و قابل کنترل‌تری هدایت کرد. این تکنیک‌ها نقش مهمی در تولید محتوای باکیفیت، از جمله در حوزه پزشکی، دارند.

⇦ **چند تکنیک پرکاربرد پرامپت‌سازی:**

۱) *پرامپت صریح (Explicit Prompting)*

در این روش، به مدل به‌صورت دقیق گفته می‌شود که چه کاری انجام دهد.

۲) *پرامپت با نقش‌دهی (Role Prompting)*

در این روش، ابتدا به مدل یک نقش یا شخصیت داده می‌شود تا رفتار خاصی داشته باشد.

۳) *پرامپت چندمرحله‌ای (Chain of Thought)*

از مدل خواسته می‌شود که مرحله‌به‌مرحله فکر کند تا به نتیجه برسد.

۴) *پرامپت با مثال (Few-shot Prompting)*

به مدل چند نمونه داده می‌شود تا با الگوگیری از آن‌ها پاسخ تولید کند.

۵) *پرامپت با محدودیت زبانی یا فرمتی*

برای کنترل ساختار خروجی، می‌توان قالب خاصی تعیین کرد (مثلاً خروجی JSON، زبان فارسی، یا لحن رسمی)

⇦ **نحوه استفاده از تکنیک‌های پرامپت‌سازی**

برای بهره‌گیری مؤثر از تکنیک‌های پرامپت‌سازی در تعامل با مدل‌های زبانی، رعایت اصول زیر ضروری است:

۱) *تعریف دقیق هدف*

در ابتدا باید مشخص شود که هدف از پرامپت چیست و مدل قرار است چه نوع خروجی‌ای تولید کند. به عنوان مثال، اگر هدف تولید داده‌ی پزشکی به زبان فارسی و در قالب JSON است، باید این موضوع به وضوح در پرامپت بیان شود.

۲) *نقش‌دهی به مدل (Role Prompting)*

با تعریف نقش مشخص برای مدل، می‌توان آن را برای تولید پاسخ‌هایی با دقت و لحن مناسب‌تر هدایت کرد.

۳) *تعیین قالب خروجی*

ضروری است که قالب دقیق مورد انتظار خروجی (مانندJSON ، جدول، لیست یا متن توصیفی) در پرامپت مشخص شود.

۴) *ارائه نمونه (Few-shot Prompting)*

با ارائه چند نمونه مشابه، مدل می‌تواند الگوی مورد نظر را بهتر درک کرده و خروجی‌های دقیق‌تری تولید کند.

⇦ **آیا الگوریتم خاصی برای بهینه‌سازی پرامپت وجود دارد؟**

بله، برای یافتن بهترین پرامپت جهت تولید خروجی‌های باکیفیت، دو رویکرد اصلی وجود دارد:

۱) *جست‌وجوی دستی (Manual Prompt Tuning)*

در این روش، کاربر با آزمون و خطا، پرامپت‌های مختلف را امتحان کرده و نتایج آن‌ها را مقایسه می‌کند تا به مؤثرترین پرامپت برسد.
این رویکرد برای تولید داده‌های محدود یا کار در محیط‌های بدون کدنویسی) مانند رابط‌های گفت‌وگومحور نظیر (ChatGPT رایج است.

۲) *بهینه‌سازی خودکار پرامپت (Automated Prompt Optimization)*

این رویکرد مبتنی بر استفاده از الگوریتم‌های نیمه‌هوشمند یا کاملاً هوشمند برای یافتن بهترین پرامپت ممکن است. برخی روش‌های متداول در این حوزه عبارتند از:

- Grid Search **یا** Random Search **بر روی** ترکیب‌های مختلف پرامپت‌ها
- یادگیری تقویتی (Reinforcement Learning - RLHF)
- الگوریتم‌های ژنتیکی (Genetic Algorithms)
- بهینه‌سازی مبتنی بر گرادیان در مدل‌های قابل تفکیک مانند Soft Prompting

پیاده‌سازی:

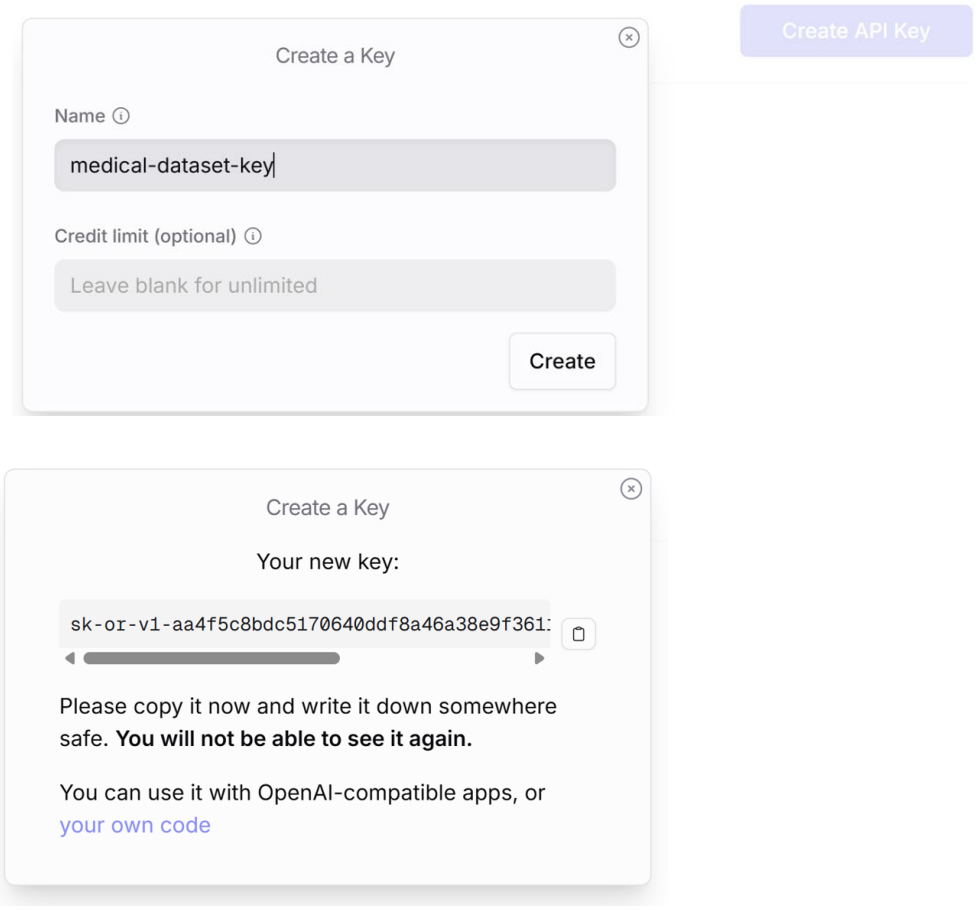
تکنیک‌های پرامپت‌سازی (Prompt Engineering) به روش‌هایی گفته می‌شود که با استفاده از آن‌ها می‌توان خروجی مدل‌های زبانی مانند ChatGPT را به‌صورت دقیق‌تر، هدفمندتر و قابل کنترل‌تری هدایت کرد. این تکنیک‌ها نقش مهمی در تولید محتوای باکیفیت، از جمله در حوزه پزشکی، دارند.

⇦ مرحله اول: دریافت و راه‌اندازی API از OpenRouter

در این مرحله از پلتفرم OpenRouter.ai استفاده کردیم، چرا که این پلتفرم امکان دسترسی به مجموعه‌ای از مدل‌های زبانی مختلف (مانند GPT-4.0، Gemini، DeepSeek و ...) را تنها با استفاده از یک API واحد فراهم می‌کند.

با وجود محدودیت‌هایی در استفاده رایگان، این ابزار برای اهداف آموزشی و پژوهشی کاملاً مناسب است. سپس به بخش API Keys مراجعه کرده و یک کلید (API Key) ایجاد کردیم تا از آن در فراخوانی مدل‌ها استفاده کنیم.

در ادامه، با مراجعه به مستندات رسمی در بخش Quickstart، نحوه‌ی استفاده از API، ساختار درخواست‌ها و پارامترهای مورد نیاز را مطالعه و در پروژه پیاده‌سازی کردیم.



لیست مدل‌هایی که می‌توان با همین API Key از آنها استفاده کرد:

مدل	سازنده	توضیح
`openai/gpt-3.5-turbo`	OpenAI	سریع و سبک، مناسب وظایف عمومی
`openai/gpt-4`	OpenAI	قوی، ولی مصرف بالا و محدودیت دارد
`openai/gpt-4-vision`	OpenAI	برای ورودی تصویری
`google/gemini-pro`	Google	مدل Gemini معمولی، پشتیبانی از فارسی، سبک‌تر
`google/gemini-pro-vision`	Google	مدل چندرسانه‌ای (متن + تصویر)
`deepseek-chat`	DeepSeek	فوق‌العاده در فهم فارسی، برای تولید دیتاست عالیه✅
`mistralai/mixtral-8x7b`	Mistral	مدل قدرتمند، مناسب متن‌سازی عمیق
`anthropic/claude-3-sonnet`	Anthropic	قوی در reasoning، زبان طبیعی، ولی انگلیسی محورتر
`meta-llama/llama-3-8b`	Meta	نسبتاً سبک، مناسب برای دیپلوی شخصی
`qwen/qwen1.5-72b-chat`	Alibaba	خیلی خوب برای سوالات چندزبانه

⇦ مرحله دوم: انتخاب مدل و استفاده از API

بین مدل‌هایی که ارائه میشه و می‌تونیم استفاده کنیم ۳ مدل اصلی وجود داره که میتونه مناسب باشه و در ادامه اینها را با هم مقایسه می‌کنیم:

ویژگی / مدل	ChatGPT (`openai/gpt-4`)	DeepSeek (`deepseek-chat`)	Gemini (`google/gemini-pro`)
پشتیبانی از زبان فارسی	خوب ولی گاهی ساختار JSON به هم می‌ریزه	عالی – تمرکز روی چینی/فارسی	متوسط – قابل قبول ولی نه عالی
محدودیت استفاده رایگان	تعداد پیام GPT-4 در روز کم (۵–۱۰)	زیاد (بیشتر از بقیه مدل‌ها)	متوسط (۱۰۰–۱۵۰ پیام روزانه)
کیفیت تولید خروجی ساختارمند (مثلاً JSON)	خیلی خوب ولی گاهی به جای JSON متن عادی می‌ده	عالی – خروجی‌های تمیز و دقیق	خوب ولی گاهی ناقص
سرعت پاسخ‌دهی	کندتر – مخصوصاً GPT-4	سریع و سبک	متوسط (پایداری بالا)
پایداری در OpenRouter	متغیر – گاهی unavailable	عالی – همیشه در دسترس	خوب – نسبتاً پایدار
ویژگی خاص	reasoning بسیار قوی	عملکرد عالی در زبان‌های آسیایی و فارسی	مدل رسمی – Google امن و شناخته‌شده
مناسب برای دیتاست پزشکی فارسی؟	خوب – برای نمونه‌سازی پیچیده	✅ بسیار مناسب	قابل قبول ولی نه دقیق‌ترین

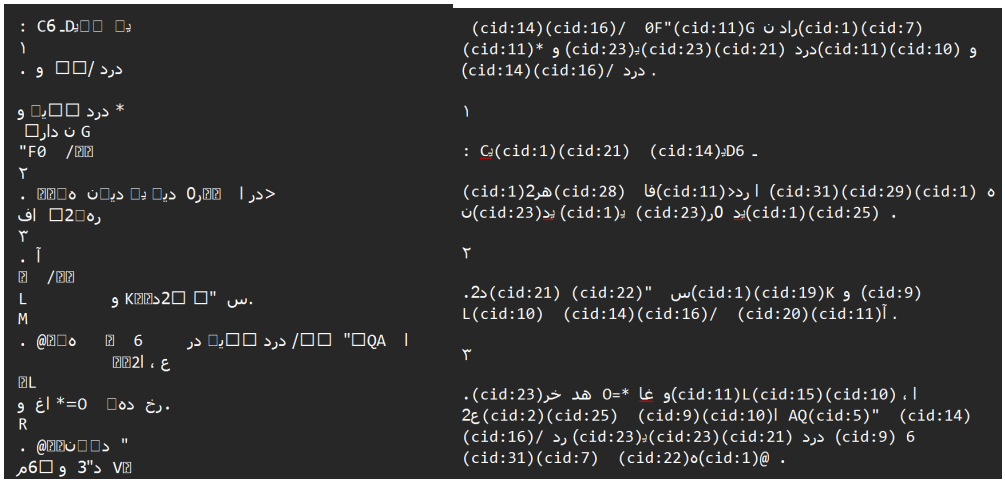
جمع‌بندی:

- برای مرحله داده‌سازی سریع و زیاد: DeepSeek
- برای مرحله مقایسه یا تست مدل‌های رسمی‌تر: Gemini
- برای مرحله تحلیل/ارزیابی پیشرفته یا تولید سوال چندگزینه‌ای با منطق قوی: GPT-4 (در حد چند نمونه)

بنابراین با DeepSeek ادامه می‌دهیم.

⇦ مرحله سوم: استخراج متن PDF

استفاده از PyMuPDF و pdfminer . six که تخصص آنها خواندن پی دی اف هست و برای فارسی قوی هستند، اما خروجی ها به ترتیب از چپ به راست نتیجه کتابخانه ها هستند:



خروجی ها دارای فونت‌های غیراستاندارد یا به‌صورت عکس/بیت‌مپ رمزگذاری‌شده هستند.

در بعضیPDF ها، حتی اگر متن رو بتوان انتخاب یا کپی کرد، باز هم ممکن است که:

- متن با فونت‌هایی ذخیره شده باشند که گلیف‌هاشون به یونیکد نگرفته‌اند.
- در واقع، PDF از فونت‌های رمزگذاری‌شده یا سفارشی (embedded fonts) استفاده کند.

بنابراین در ادامه از روش راه‌اندازی OCR با Tesseract استفاده می‌کنیم.

چون متن قابل استخراج نیست، باید PDF رو به تصویر تبدیل کنیم و با OCR متن رو **بازشناسی نوری** کنیم.

۱. تبدیل صفحات PDF به تصویر (با `pdf2image`)
۲. نصب و تنظیمTesseract (برای OCR فارسی)
۳. پردازش هر صفحه با `pytesseract`

ابزارهایی که نیاز داریم:

- کتابخانه های pdf2image و pytesseract برای به ترتیب تبدیل PDF به عکس و OCRروی تصاویر.
- نصب Tesseract OCR بسته نرم‌افزاری اصلی + OCR زبان فارسی

استفاده از لینک زیر برای دانلود نرم افزار:

https://github.com/UB-Mannheim/tesseract/wiki

و دانلود fas.traineddata که برای زبان فارسی ست و قرار دادن آن در فولدر tessdata.

از کد زیر برای تست کردن استفاده کردیم:

```python
from pdf2image import convert_from_path
import pytesseract
from PIL import Image

# Set the path to Tesseract executable (only needed on Windows)
pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"

# Convert the first page of the PDF to an image
images = convert_from_path("Book-1-120.pdf", dpi=300)
first_page = images[0]

# Save the image for preview (optional)
first_page.save("page1.jpg", "JPEG")

# Perform OCR on the image using Persian (Farsi) language
text = pytesseract.image_to_string(first_page, lang='fas')

# Print the extracted text
print(text)

# Save the OCR result to a text file
with open("ocr_output.txt", "w", encoding="utf-8") as f:
    f.write(text)
```

نتیجه (و همینطور تصویر JPEG ذخیره شده هم در کنار فایل ها موجود است):

```
درد چشم و سردرد شدید و ضربان‌دار قرمزی چشم ۱۰.

تاری دید یا دیدن هاله در اطراف نورها ۲

کره چشم سفت و حساس می‌شود ۲.

گاهی به علت درد شدید در چشم ممکن است تهوع, استفراغ و ضعف رخ دهد ۴.

گشادشدن مردمک و عدم پاسخ آن به نور (مردمک در حالت عادی در اثر تابش نور تنگ می‌شود) ۵.

علل بیماری:

عوامل افزایش دهنده خطر::
سابقه خانوادگی آب سیاه یا دوربینی ۲

استفاده از بعضی از داروها که باعث مهار سیستم کولینرژیک (پاراسمپاتیک) می‌شوند ۲.
```

بررسی نتیجه:

از نظر کیفیت OCR فارسی نتیجه خوبی دارد چراکخ، ساختار متن حفظ شده (تیترها، شماره‌گذاری، ترتیب بخش‌ها) و کلمات فارسی با دقت بسیار بالا تشخیص داده شده‌اند.

⇦ مرحله چهارم: استخراج ساختارمند اطلاعات بیماری‌ها

آماده سازی و مرتب سازی فایل متنی برای پرامپت نویسی

- **حذف حروف انگلیسی**
- **تبدیل اعداد انگلیسی به فارسی**
  تشخیص مرز بین بیماری‌ها
- **حذف نویز اطراف مرزها**
- **استخراج جفت اسامی فارسی و انگلیسی بیماری ها از ورژن OCR fas+eng**
- **جایگزینی جفت اسامی فارسی و انگلیسی بیماری ها به ورژن فقط فارسی:**

معادل های جفت اسامی را از ورژن ترکیبی انگلیسی و فارسی برداشتیم، و کلمات انگلیسی آنها را در عنوان قرار دادیم و برای قسمت فارسی از کلمات فارسی برداشته شده از ورژن فقط فارسی استفاده کردیم، چرا که در ورژن ترکیبی اسامی فارسی با دقت ورژن فقط فارسی استخراج نشده بودند، اینکار باعث شد متن خروجی بصورت خیلی خوبی تمیز شود، و عنوان های فارسی و انگلیسی به درستی قرار بگیرند.

Text Cleaning

```
with open("ocr_all_pages.txt", "r", encoding="utf-8") as f:
    text = f.read()

text = re.sub(r"--- Page \d+ ---", "", text)
```
✓ 0.0s

```
text = re.sub(r"[a-zA-Z]+", "", text)
```
✓ 0.0s

```
def convert_english_digits_to_persian(text):
    english_to_persian = str.maketrans("0123456789", "۰۱۲۳۴۵۶۷۸۹")
    return text.translate(english_to_persian)

text = convert_english_digits_to_persian(text)
```
✓ 0.0s

```
text = text.replace("//", "\n--- بیماری ---\n")
```
✓ 0.0s

```
text = re.sub(r"\n{2,}", "\n\n", text)
```
✓ 0.0s

فایل های موجود در گزارش و اطلاعات هرکدام:

- raw_text.txt:
  استخراج متن از پی دی اف با کتابخانه pdf2image
- miner_output.txt:
  استخراج متن از پی دی اف با کتابخانه pytesseract
- ocr_all_pages_eng_fas.txt:
  خروجی تمام صفحات با ورژن fas+eng OCR
- ocr_all_pages.txt:
  خروجی تمام صفحات با ورژن fas OCR
- ocr_all_pages_cleaned.txt:
  خروجی فایل تمیز شده ی ocr_all_pages
- ocr_all_pages_paired_names.txt:
  خروجی فایل ocr_all_pages_cleaned با اضافه کردن نام اسامی انگلیسی
- ocr_disease_final.txt:
  خروجی فایل ocr_all_pages_paired_names با جایگزینی جفت اسامی فارسی و انگلیسی بصورت مرتب شده نهایی

⇦ مرحله پنجم: تولید داده مصنوعی با طراحی پرامپت هدفمند (Prompt Engineering)

نکاتی که باید رعایت کنیم:

بخش	توضیح
symptom	باید توصیفی از سن، جنسیت، علائم ظاهری بیمار، و زمینه مراجعه باشه
Diagnosis	فقط نام بیماری یا تشخیص نهایی مدل یا پزشک

نوشتن پرامپت به گونه‌ای که:

- مدل متوجه بشه قراره یک نمونه سناریو پزشکی واقعی بسازه.
- در خروجی فقط JSON شامل symptom و diagnosis بده.
- محتوای خروجی باید بر اساس اطلاعاتی باشه که ما از PDF استخراج کردیم.

به دلیل محدودیت توکن های ورودی (44000) بخش هایی از فایل تکست ورودی را حذف کردیم.

پرامپت های محتلفی را تست کردیم و جمله بندی زیر نتایج قابل قبولی داد.



از مدل DeepSeek استفاده کردیم:

```python
# OpenRouter API key
api_key = "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"

# API endpoint and headers
url = "https://openrouter.ai/api/v1/chat/completions"
headers = {
    "Authorization": f"Bearer {api_key}",
    "Content-Type": "application/json"
}
```

```python
data = {
    "model": "deepseek/deepseek-chat",
    "messages": [
        {
            "role": "user",
            "content": prompt
        }
    ]
}

# Send request and print the response
response = requests.post(url, headers=headers, json=data)
```

```python
if response.status_code == 200:
    result = response.json()
    output = result["choices"][0]["message"]["content"]

    try:
        # Save the raw model output to a text file (no parsing, no printing)
        with open("model_output_raw.txt", "w", encoding="utf-8") as f:
            f.write(output)

        print("Output saved to model_output_raw.txt")

    except Exception as e:
        print("Error saving the output:", e)


else:
    # Report HTTP error from the API response
    print("Error fetching response:", response.status_code, response.text)
```

ساختار کد:

۱) بارگذاری داده اولیه (اطلاعات بیماری‌ها)
۲) تعریف نمونه خروجی
۳) ساخت پرامپت نهایی
۴) ارسال درخواست به API : با استفاده از کلید API و تنظیمات HTTP ، پرامپت به مدل ارسال می‌شود و پاسخ آن دریافت می‌گردد.
۵) ذخیره خروجی خام مدل در فایل

نمونه خروجی:

```
[
  {
    "symptom": "مردی ۴۵ ساله با درد شدید چشم و سردرد مراجعه کرده است. چشم وی قرمز و سفت است.",
    "diagnosis": ["آب سیاه حاد چشم", "Acute Angle-Closure Glaucoma"]
  },
  {
    "symptom": "زنی ۶۰ ساله با تاری دید و دیدن هاله در اطراف نورها به پزشک مراجعه کرده است.",
    "diagnosis": ["آب سیاه چشم، نوع مزمن زاویه باز", "Chronic Open-Angle Glaucoma"]
  },
  {
    "symptom": "مردی ۷۰ ساله با دید مه‌آلود و دید تار در نور روشن به چشم پزشک مراجعه کرده است.",
    "diagnosis": ["آب مروارید", "Cataract"]
  },
  {
    "symptom": "مردی ۳۵ ساله با درد شدید در ناحیه مقعد و تب به اورژانس مراجعه کرده است.",
    "diagnosis": ["آبسه آنورکتال", "Anorectal Abscess"]
  },
  {
    "symptom": "زنی ۲۸ ساله با درد و قرمزی در سینه‌اش به پزشک مراجعه کرده است. وی تب دارد.",
    "diagnosis": ["آبسه پستان", "Breast Abscess"]
  },
```

و در انتها فایل داده‌ها را به JSON تبدیل می‌کنیم.

```python
# Read the raw output text from the file
with open("model_output_raw.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

try:
    # Try to parse the raw text into valid JSON
    data = json.loads(raw_text)

    # Save the parsed data to a proper JSON file
    with open("generated_samples.json", "w", encoding="utf-8") as out_file:
        json.dump(data, out_file, indent=2, ensure_ascii=False)

    print("Successfully converted and saved to 'generated_samples.json'.")
except Exception as e:
    # Handle and print any parsing error
    print("Error converting the file:", e)
```

نمونه پرامپت های دیگه ی استفاده شده:

یکی از مشکلات مهم در ابتدا عدم تنوع در جملات بود، و سپس مشکل اصلی تولید نکردن دقیقا ۱۰۰ نمونه بود.

```
prompt = (
    "شما یک مدل تولید داده هستید و باید با استفاده از اطلاعات پزشکی، نمونه‌های مصنوعی بسازید.\n"
    "شروع شده‌اند \"--- بیماری ---\" در فایل زیر، چندین بیماری داریم که هرکدام با خط.\n"
    "با توجه به این اطلاعات، ۱۰۰ نمونه بیمار واقعی‌نما تولید کنید.\n"
    "هر نمونه شامل این دو بخش باشد:\n"
    "- \"symptom\": شرح حال بیمار با علائم مشخص‌کننده (سن، جنسیت، نشانه‌ها)\n"
    "- \"diagnosis\": لیستی شامل [\"نام فارسی بیماری\", \"نام انگلیسی\"]\n\n"
    "الزامات مهم:\n"
    "- خروجی فقط یک آرایه JSON باشد نمونه با دقیقا ۱۰۰.\n"
    "- هیچ متن اضافی قبل یا بعد از JSON نوشته نشود.\n"
    "- تنوع جمله‌بندی رعایت شود، اما ساختار حفظ شود.\n"
    "- اگر کمتر یا بیشتر از ۱۰۰ نمونه باشد، خروجی نادرست تلقی می‌شود.\n\n"
    + example +
    "\nمتن بیماری‌ها:\n" + disease_text
)
```

با تغییرات مختلف روی پرامپت همچنان به علت زیاد بودن حجم توکن ها تعداد را بدرستی نمی‌ساخت.

DeepSeek (مثل بسیاری از مدل‌های مشابه) محدودیت در **تعداد توکن‌های ورودی + خروجی** دارد. اگر:

- متن `disease_text` طولانی باشد
- به آن پرامپت سنگین اضافه شده باشد
- بخواهیم ۱۰۰ خروجی کامل JSON هم تولید شود

مدل یا اصلاً شروع به تولید نمی‌کند، یا زودتر از اتمام، خروجی را قطع می‌کند.

برای همین ۲ بار پرامپت با درخواست سازخت ۵۰ نمونه ارسال کردیم و با تعداد مناسب ۵۰ تا ۲ تا نتیجه را با هم ادغام کردیم.

۲. گام دوم: تنظیم مدل روی دیتاست ساخته شده

از مدل Llama-3.2-1B به عنوان Base Model استفاده کرده و آن را روی داده تولیدشده در مرحله قبل، به کمک تکنیک LoRA تنظیم (fine-tune) نمایید.

مدل نهایی را به عنوان یک مدل جدید ذخیره و مستندات مربوط به آموزش آن (پارامترهای آموزش، تنظیمات LoRA، کتابخانه‌ها) را در گزارش ارائه دهید.

توضیحات اولیه:

⇦ **مدل LLaMA-3.2-1B چیست؟**

- LlaMA مخفف «Large Language Model Meta AI» است و توسط شرکت Meta (فیسبوک سابق) طراحی شده.
- عدد 3.2-1B به این معنی است که ما از نسخه‌ی سوم این مدل استفاده می‌کنیم که حدود ۱ میلیارد پارامتر دارد.
- این مدل از نوع Decoder-only Transformer است که مناسب کارهای زبانی مانند تولید متن، خلاصه‌سازی، پاسخ‌گویی و... است.
- LLaMA 3 نسبت به نسخه‌های قبلی دقت بالاتری دارد و با مجموعه‌داده‌های متنوع‌تری آموزش دیده.

⇦ **تکنیک LoRA چیست؟**

LoRA (Low-Rank Adaptation) یک روش سبک و بهینه برای تنظیم مدل‌های بزرگ است:

- به‌جای تنظیم کل پارامترهای مدل (که بسیار پرهزینه است)، LoRA فقط چند لایه‌ی جدید (low-rank) را به مدل اضافه می‌کند.
- در این روش، **پارامترهای اصلی فریز می‌شوند** و فقط لایه‌های کوچک اضافه‌شده آموزش می‌بینند.
- این تکنیک باعث می‌شود که:
  - حجم مدل آموزش‌دیده‌شده خیلی کم باشد.
  - مصرف منابع و GPU پایین بیاید.
  - به‌راحتی بتوان آن را به مدل اصلی اضافه یا جدا کرد.

پیاده‌سازی:

برای استفاده از مدل باید از گوگل کولب استفاده کنیم، مدل های لاما برای دسترس عموم قرار داده نشده، بنابراین از وبسایت زیر به بخش کولب میرویم و ادامه کار را آنجا انجام میدهیم:

https://huggingface.co/unsloth/Llama-3.2-1B

⇦ **مرحله اول: اجرای کدهای آماده تعریف شده در گوگل کولب**

۱) نصب کتابخانه‌های مورد نیاز:

این قسمت مربوط به نصب وابستگی‌ها (dependencies) هست که لازمه برای اجرای آموزش با Unsloth و LoRA روی Google Colab آماده باشد.

Installation

```
%%capture
import os
if "COLAB_" not in "".join(os.environ.keys()):
    !pip install unsloth
else:
    # Do this only in Colab notebooks! Otherwise use pip install unsloth
    !pip install --no-deps bitsandbytes accelerate xformers==0.0.29.post3 peft trl==0.15.2 triton cut_cross_entropy unsloth_zoo
    !pip install sentencepiece protobuf "datasets>=3.4.1" huggingface_hub hf_transfer
    !pip install --no-deps unsloth
```

❖ *ابزارهای اصلی:*

- Unsloth: کتابخانه‌ی اصلی برای آموزش سریع با LLaMA و LoRA
- Peft: پیاده‌سازی LoRA (برای فاین‌تیون سبک)
- Bitsandbytes: اجرای مدل‌ها به صورت ۸bit- (کاهش مصرف رم)
- xformers, triton: بهینه‌سازی سرعت و حافظه در عملیات مدل
- datasets, huggingface_hub: مدیریت و بارگذاری دیتاست و مدل‌ها

❖ *جمع‌بندی:*

این مرحله همه‌چیز را آماده می‌کند برای:

- آموزش مدل LLaMA با حافظه‌ی کم
- استفاده از تکنیک LoRA
- اتصال به Hugging Face
- پشتیبانی از کوئری‌های طولانی یا دیتاست‌های بزرگ

۲) بارگذاری مدل LLaMA با کمک کتابخانه‌ی Unsloth:

```
from unsloth import FastLanguageModel
import torch
max_seq_length = 2048 # Choose any! We auto support RoPE Scaling internally!
dtype = None # None for auto detection. Float16 for Tesla T4, V100, Bfloat16 for Ampere+
load_in_4bit = True # Use 4bit quantization to reduce memory usage. Can be False.

# 4bit pre quantized models we support for 4x faster downloading + no OOMs.
fourbit_models = [
    "unsloth/Meta-Llama-3.1-8B-bnb-4bit",      # Llama-3.1 15 trillion tokens model 2x faster!
    "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit",
    "unsloth/Meta-Llama-3.1-70B-bnb-4bit",
    "unsloth/Meta-Llama-3.1-405B-bnb-4bit",    # We also uploaded 4bit for 405b!
    "unsloth/Mistral-Nemo-Base-2407-bnb-4bit", # New Mistral 12b 2x faster!
    "unsloth/Mistral-Nemo-Instruct-2407-bnb-4bit",
    "unsloth/mistral-7b-v0.3-bnb-4bit",        # Mistral v3 2x faster!
    "unsloth/mistral-7b-instruct-v0.3-bnb-4bit",
    "unsloth/Phi-3.5-mini-instruct",           # Phi-3.5 2x faster!
    "unsloth/Phi-3-medium-4k-instruct",
    "unsloth/gemma-2-9b-bnb-4bit",
    "unsloth/gemma-2-27b-bnb-4bit",            # Gemma 2x faster!
] # More models at https://huggingface.co/unsloth

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Meta-Llama-3.1-8B",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
    # token = "hf_...", # use one if using gated models like meta-llama/Llama-2-7b-hf
)
```

❖ *تنظیمات اولیه:*

- max_seq_length: حداکثر طول دنباله‌ای که مدل می‌تواند بخواند. (مثلاً ۲۰۴۸ توکن).
- load_in_4bit: بارگذاری سبک‌تر با استفاده از 4-bit quantization، مصرف رم خیلی پایین‌تر و سریع‌تر.

❖ *لیست مدل‌ها*

❖ *بارگزاری مدل و tokenizer با تنظیمات قبلی:*

فایل‌های زیر با موفقیت دانلود شدند:

- model.safetensors: وزن‌های مدل
- tokenizer_config.json, tokenizer.json, special_tokens_map.json: تنظیمات و نقشه توکن‌ها
- generation_config.json: تنظیمات پیش‌فرض برای تولید متن

❖ *جمع‌بندی:*

الان مدل LLaMA آماده است و روی GPU با تنظیمات سبک و سریع بارگذاری شده است.

۳) بارگذاری و پیکربندی LoRA adapters روی مدل اصلی:

We now add LoRA adapters so we only need to update 1 to 10% of all parameters!

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Choose any number > 0 ! Suggested 8, 16, 32, 64, 128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 16,
    lora_dropout = 0, # Supports any, but = 0 is optimized
    bias = "none",    # Supports any, but = "none" is optimized
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch sizes!
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for very long context
    random_state = 3407,
    use_rslora = False,  # We support rank stabilized LoRA
    loftq_config = None, # And LoftQ
)
```

```
Unsloth 2024.8 patched 32 layers with 32 QKV layers, 32 O layers and 32 MLP layers.
```

- **r** (rank): عمق لایه LoRA. هر چی بزرگ‌تر باشه، ظرفیت یادگیری بیشتر ولی مصرف RAM و زمان هم بالاتر.
- **target_modules**: لیست ماژول‌هایی که LoRA روی اون‌ها اعمال میشه. اینها در واقع اجزای اصلی مدل ترنسفورمر هستند (Q, K, V, etc.)، معمولاً همین مجموعه برای مدل‌های LLaMA استفاده میشه.
- **lora_alpha**: ضریب تقویتی برای تنظیم خروجی LoRA. این عدد معمولاً مساوی با r تنظیم میشود.
- **lora_dropout**: میزان Dropout در لایه LoRA. LoRA چون سبک و ظریف است، معمولاً 0 انتخاب بهینه‌ایه.

❖ *جمع‌بندی:*

در این مرحله، مدل آماده شده تا فقط وزن‌های کم‌حجم و خاص (LoRA adapters) در طول آموزش تغییر کنن. این تکنیک باعث میشه هم مصرف VRAM و هم زمان آموزش به‌شدت کاهش پیدا کنه، بدون اینکه کیفیت خیلی افت کنه.

۴) آماده‌سازی داده‌های ورودی:

توی این بخش، داده‌هایی که در گام اول، تولید کردیم، یعنی ۱۰۰ نمونه‌یمان را آپلود می‌کنیم.

```
[6]  from google.colab import files
     uploaded = files.upload()
```



Choose Files generated_samples.json

- **generated_samples.json**(application/json) - 19769 bytes, last modified: 5/20/2025 - 100% done

Saving generated_samples.json to generated_samples.json

```
[7]  import json
     from datasets import Dataset

     with open("generated_samples.json", "r", encoding="utf-8") as f:
         data = json.load(f)

     dataset = Dataset.from_list(data)
```

در ادامه مجموعه داده‌یمان را به فرمت Alpaca تبدیل می‌کنیم. فرمت **Alpaca-like** یکی از رایج‌ترین قالب‌ها برای آموزش مدل‌های زبان مثل LLaMA با ساختار **دستور (instruction)**، **ورودی (input)**، و **پاسخ (response)** است.

ساختار دقیق Alpaca-like format همراه با مثال:

```
### Instruction:
(توضیح وظیفه یا سوالی که کاربر دارد)

### Input:
(اطلاعات ورودی – می‌تواند خالی باشد)

### Response:
(پاسخ مدل به آن سوال یا ورودی)
```

```
### Instruction:
علائم زیر مربوط به کدام بیماری است؟

### Input:
مردی ۴۵ ساله با درد شدید چشم و سردرد مراجعه کرده است. چشم وی قرمز و سفت است.

### Response:
آب سیاه حاد چشم (Acute Angle-Closure Glaucoma)
```

- این ساختار به مدل آموزش می‌دهد چطور به ورودی‌های خاص پاسخ دهد.
- ساختار سه‌بخشی باعث می‌شود مدل بهتر تفکیک کند که کدام قسمت پرسش است و کدام قسمت پاسخ.
- برای اکثر مدل‌های instruction-tuned مثلLLaMA ، Mistral، Zephyrو ... همین فرمت به عنوان داده آموزشی استفاده می‌شود.

```
alpaca_prompt = """### Instruction:
بر اساس علائم و شکایات زیر، تشخیص بیماری را بده.

### Input:
{}

### Response:
{}"""

def format_dataset(example):
    instruction = "بر اساس علائم و شکایات زیر، تشخیص بیماری را بده."
    input_text = example["symptom"]
    output_text = f"{example['diagnosis'][0]} ({example['diagnosis'][1]})"  #fas+eng
    text = alpaca_prompt.format(input_text, output_text)
    return {"text": text}

# converting the dataset to Alpaca format
dataset = dataset.map(format_dataset)
```

```
Map: 100% 83/83 [00:00<00:00, 1599.27 examples/s]
```

اکنون بعد از dataset.map(...)، خروجی Dataset حاوی فیلد "text" است که ترکیبی از دستور، ورودی و خروجی به فرمت مناسب برای فاین‌تیون است و فرمتی مشابه زیر را دارد:

```
{'symptom': 'مردی ۴۵ ساله با درد شدید چشم و سردرد مراجعه کرده است. چشم وی قرمز و سفت است.', 'diagnosis': ['آب سیاه حاد چشم', 'Acute Angle-Closure Glaucoma'], 'text': '### Instruction:\nبر اساس علائم و شکایات زیر، تشخیص بیماری را بده.\n\n### Input:\nمردی ۴۵ ساله با درد شدید چشم و سردرد مراجعه کرده است. چشم وی قرمز و سفت است.\n\n### Response:\nآب سیاه حاد چشم (Acute Angle-Closure Glaucoma)'}
```

۵) آموزش مدل Llama با داده‌های بارگزاری شده:

❖ *راه اندازی تنظیمات مدل برای آموزش*

این قسمت مربوط به راه‌اندازی مرحله آموزش (training) مدل با استفاده از کلاس SFTTrainer از کتابخانه‌ی trl هست.

SFTTrainer(...) یک کلاس از trl برای "Supervised Fine-Tuning" طراحی شده، یعنی آموزش با نظارت بر پایه داده‌هایی که خروجی (label) مشخص دارند، مثل قالب Alpaca که استفاده کردیم.

```
[ ]  from trl import SFTTrainer
     from transformers import TrainingArguments
     from unsloth import is_bfloat16_supported

     trainer = SFTTrainer(
         model = model,
         tokenizer = tokenizer,
         train_dataset = dataset,
         dataset_text_field = "text",
         max_seq_length = max_seq_length,
         dataset_num_proc = 2,
         packing = False, # Can make training 5x faster for short sequences.
         args = TrainingArguments(
             per_device_train_batch_size = 2,
             gradient_accumulation_steps = 4,
             warmup_steps = 5,
             # num_train_epochs = 1, # Set this for 1 full training run.
             max_steps = 60,
             learning_rate = 2e-4,
             fp16 = not is_bfloat16_supported(),
             bf16 = is_bfloat16_supported(),
             logging_steps = 1,
             optim = "adamw_8bit",
             weight_decay = 0.01,
             lr_scheduler_type = "linear",
             seed = 3407,
             output_dir = "outputs",
             report_to = "none", # Use this for WandB etc
         ),
```

- ورودی‌های SFTTrainer:

- `model, tokenizer`

مدل و tokenizer که قبلاً بارگذاری شده بودند.

- `train_dataset = dataset`

دیتاست آموزش که در فرمت Alpaca آماده کردید.

- `dataset_text_field = "text"`

مشخص می‌کند که فیلد متنی مورد استفاده در داده‌ها، فیلد `"text"` هست. (دقیقاً همان فیلد تولید شده).

- `max_seq_length = max_seq_length`

حداکثر طول توکن‌هایی که در یک نمونه ورودی در نظر گرفته می‌شن. مقدارش قبلاً ۲۰۴۸ تعیین شده بود.

- `dataset_num_proc = 2`

تعداد هسته‌هایی که برای پیش‌پردازش (مثل tokenization) استفاده می‌شه — موازی‌سازی سرعت رو افزایش می‌دهد.

- `packing = False`

اگر `True` باشه، داده‌های کوتاه‌تر رو به‌صورت متوالی در یک توکن‌سازی می‌چسبونه. در حالت `False` یعنی هر نمونه مستقل پردازش می‌شه. برای داده‌های آموزشی مثل ما بهتره `False` باشه.

- تنظیمات آموزش (TrainingArguments(...))

- `per_device_train_batch_size = 2`

تعداد نمونه‌ها در هر batch برای هر GPU

- `gradient_accumulation_steps = 4`

مدل به‌جای به‌روزرسانی وزن‌ها بعد از هر batch، تا ۴ batch صبر می‌کنه و بعد گرادیان‌ها رو به‌طور تجمیعی به‌روزرسانی می‌کنه.

- *warmup_steps = 5*

تعداد قدم‌هایی که مدل به آرامی یادگیری رو شروع می‌کنه (برای جلوگیری از نوسان شدید).

- *max_steps = 60*

مدل فقط ۶۰ مرحله آموزش می‌بینه (برای تست سریع) – (برای آموزش کامل num_train_epochs = 1)

- *learning_rate = 2e-4*

نرخ یادگیری.

- *bf16, bfl16*

آیا از float16 یا bfloat16استفاده کنه بر اساس GPU.

- *logging_steps = 1*

هر چند مرحله لاگ گرفته بشه — اینجا هر مرحله.

- *optim = "adamw_8bit"*

بهینه‌سازی با آدم‌دبلیو نسخه‌ی ۸بیتی (مصرف حافظه پایین‌تر).

- *weight_decay = 0.01*

برای جلوگیری از overfitting

- *lr_scheduler_type = "linear"*

مدل به آرامی نرخ یادگیری رو از مقدار اولیه کم می‌کنه.

- *output_dir = "outputs"*

خروجی مدل آموزش‌دیده توی این پوشه ذخیره می‌شه.

۲ بخش زیر را تغییر دادیم:

بخش	تغییر	هدف
num_train_epochs	از مقدار پیش‌فرض به 1 تغییر کرد.	آموزش کامل روی کل دیتاست فقط یک بار
max_steps	حذف شد.	غیرفعال‌سازی محدودیت مرحله‌ای برای جلوگیری از قطع شدن آموزش

از num_train_epochs استفاده کردیم بخاطر اینکه:

- با epoch مدل دقیقاً می‌فهمه چند بار باید **کل دیتاست رو ببینه**
- در داده‌های کم، مهمه که هر نمونه چند بار دیده بشه
- استفاده از max_steps برای داده‌ی کم کنترل‌پذیر نیست و ممکنه بعضی نمونه‌ها زیاد دیده نشن

در اینصورت تعداد stepها برابر مقدار زیر است:

*steps_per_epoch = total_samples / (batch_size * gradient_accumulation_steps)*

*total_steps = steps_per_epoch * num_train_epochs*

❖ *اجرا و شروع آموزش:*

ابتدا با مقدار num_train_epochs = 1 مدل را آموزش دادیم:

```
trainer_stats = trainer.train()
```

```
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 83 | Num Epochs = 1 | Total steps = 10
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
 "-____-"     Trainable parameters = 41,943,040/8,000,000,000 (0.52% trained)
                                                [10/10 00:34, Epoch 0/1]
```

Step	Training Loss
1	2.691600
2	2.772100
3	2.705300
4	2.229600
5	2.080400
6	1.665400
7	1.386100
8	1.114900
9	0.853300
10	0.823100

سپس با مقدار num_train_epochs = 5 مدل را آموزش دادیم، و ۲ بار آن را تکرار کردیم، که مجموعه نتایج آن به شرح زیر بود:

نتایج بار اول:

Step	Training Loss	18	0.760700	36	0.432900	53	0.303500
1	2.973600	19	0.602400	37	0.364800	54	0.313700
2	2.893400	20	0.550200	38	0.431000	55	0.249400
3	2.853800	21	0.551000	39	0.302200	56	0.250500
4	2.528300	22	0.527800	40	0.376400	57	0.239300
5	2.256400	23	0.447600	41	0.285800	58	0.239900
6	2.004100	24	0.471600	42	0.335000	59	0.206100
7	1.542700	25	0.641400	43	0.265900	60	0.332200
8	1.253800	26	0.501700	44	0.315000		
9	1.104600	27	0.462700	45	0.288500		
10	0.908200	28	0.452400	46	0.370900		
11	0.872500	29	0.389400	47	0.253700		
12	0.906000	30	0.354400	48	0.339100		
13	0.804700	31	0.428800	49	0.364800		
14	0.784100	32	0.467800	50	0.354700		
15	0.732500	33	0.417300	51	0.321200		
16	0.777900	34	0.348800	52	0.287200		
17	0.634900	35	0.444000	53	0.303500		

نتایج بار دوم:

Step	Training Loss	18	0.315700	36	0.230900
1	0.270600	19	0.278500	37	0.204500
2	0.245200	20	0.273600	38	0.206100
3	0.279900	21	0.277400	39	0.211800
4	0.290900	22	0.255100	40	0.177600
5	0.280100	23	0.242000	41	0.172800
6	0.303200	24	0.218600	42	0.180900
7	0.334000	25	0.279600	43	0.150100
8	0.290100	26	0.211700	44	0.176600
9	0.305400	27	0.183100	45	0.160100
10	0.282700	28	0.183800	46	0.194000
11	0.294900	29	0.194500	47	0.182700
12	0.328500	30	0.209300	48	0.184700
13	0.300400	31	0.223900	49	0.189800
14	0.225100	32	0.247000	50	0.186200
15	0.225900	33	0.201900	51	0.167400
16	0.269100	34	0.165500	52	0.171800
17	0.278200	35	0.276200	53	0.151500

53	0.151500
54	0.159100
55	0.135400
56	0.145000
57	0.165700
58	0.137700
59	0.145600
60	0.156300

❖ یک *inference/test* تکی

با توجه به نوع داده‌های خودمان این بخش را اصلاح کرده و پیاده سازی می‌کنیم.

```
# Define the instruction-based prompt
alpaca_prompt = """### Instruction:
بر اساس علائم و شکایات زیر، تشخیص بیماری را بده.

### Input:
{}

### Response:
"""

# Enable inference mode (optimized for unsloth models)
FastLanguageModel.for_inference(model)

# Example symptom input
symptom_input = "مردی ۴۰ ساله با سرفه خشک و کاهش وزن مراجعه کرده است."

# Tokenize the prompt with your symptom
inputs = tokenizer(
    [alpaca_prompt.format(symptom_input)],
    return_tensors="pt"
).to("cuda")

# Generate prediction
outputs = model.generate(**inputs, max_new_tokens=64, use_cache=True)

# Decode and print result
print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
```

- **تعریف قالب prompt (Alpaca-style)**
  شامل ### Instruction, ### Input, و ### Response:
- **فعال‌سازی حالت inference در مدل**
  برای بهینه‌سازی حافظه و سرعت در
- **وارد کردن یک نمونه‌ی تستی (symptom)**: "مردی ۴۰ ساله با سرفه خشک و کاهش وزن"...
- **جایگذاری ورودی در قالب prompt**
  یعنی ساخت متن نهایی‌ای که مدل باید پاسخ بده
- **تبدیل متن prompt به توکن (tokenization)**
  برای اینکه مدل بتونه پردازشش کنه
- **تولید پاسخ توسط مدل (generate)**
  یعنی پیش‌بینی ادامه‌ی متن توسط LLaMA
- **تبدیل توکن خروجی به متن و نمایش آن**
  نمایش تشخیص بیماری پیش‌بینی‌شده توسط مدل

خروجی:

```
### Instruction:
بر اساس علائم و شکایات زیر، تشخیص بیماری را بده.

### Input:
مردی ۴۰ ساله با سرفه خشک و کاهش وزن مراجعه کرده است.

### Response:
سرطان ریه (Lung Cancer)
```

که همانطور هم که می‌بینیم، نتیجه برای این نمونه صحیح است.

❖ *ذخیره بهترین وزن‌های مدل:*

در ادامه بهترین وزن‌های مدل را ذخیره می‌کنیم.

```
model.save_pretrained("lora_model")  # Local saving
tokenizer.save_pretrained("lora_model")
```

❖ *تست مدل با حالت real-time:*

اجرای **مرحله inference** روی مدل fine-tuned شده، برای دریافت پاسخ مدل نسبت به یک ورودی واقعی (علامت بالینی بیمار) و مشاهدهٔ خروجی زنده.(real-time)

ابتدا مدل آموزش دیده رو از حافظه (که قبل تر ذخیره کردیم) بارگزاری می‌کنیم و سپس از کد بعدیش استفاده می‌کنیم:

```
if True:
    from unsloth import FastLanguageModel
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name = "lora_model", # YOUR MODEL YOU USED FOR TRAINING
        max_seq_length = max_seq_length,
        dtype = dtype,
        load_in_4bit = load_in_4bit,
    )
    FastLanguageModel.for_inference(model) # Enable native 2x faster inference
```

```
from transformers import TextStreamer
text_streamer = TextStreamer(tokenizer)
_ = model.generate(**inputs, streamer=text_streamer, max_new_tokens=128)
```

نتیجه ۲ تا کد بالا مثل حالت قبل خواهد بود، اما همزمان موقع اجرا، مراحل هم به ترتیب real-time نمایش داده می‌شوند.

۳. گام سوم: ارزیابی عملکرد مدل جدید

برای ارزیابی مدل جدید، با استفاده از پرامپت مناسب و ChatGPT ، یک دیتاست مصنوعی به فرم سوالات چهارگزینه‌ای همراه با پاسخ درست (MCQs) تولید کنید:

نمونه:

```
[
 {
  "question": "کدام یک از موارد زیر از عوارض شایع آبله مرغان است؟",
  "options": [
   "پانکراتیت",
   "پنومونی",
   "هپاتیت ویروسی",
   "فلج اطفال"
  ],
  "answer": "پنومونی"
 },
]
```

برای این دیتاست نیز ۱۰۰ سوال و جواب تولید شود و به فرمت JSON باشد.

معیار ارزیابی دقت (Accuracy) می‌باشد.

با استفاده از پرامپت مناسب، خروجی مدل را روی دیتاست ارزیابی، تولید کنید و سپس با جواب درست در دیتاست مقایسه کنید و دقت را محاسبه کنید.

پیاده‌سازی:

- *تولید داده‌های تست:*

ابتدا باید با پرامپت مناسب داده‌های تست را تولید کنیم.

در وهله اول از پرامپت زیر استفاده کردیم، که نتایج آن را هم در ادامه می‌گذاریم:

```
prompt = (
    "شما یک مدل تولید داده تستی برای ارزیابی مدل‌های هوش مصنوعی هستید.\n"
    "وظیفه شما تولید 50 سؤال چندگزینه‌ای (MCQ) بر اساس متن داده‌شده است.\n"
    "هر نمونه باید شامل موارد زیر باشد:\n"
    "- \"question\": یک پرسش واضح و دقیق درباره یکی از بیماری‌ها، علائم یا عوارض آن\n"
    "- \"options\": چهار گزینه‌ی ممکن (فقط یک گزینه درست باشد، سایر گزینه‌ها باید منطقی اما نادرست باشند)\n"
    "- \"answer\": متن گزینه صحیح که دقیقاً یکی از چهار گزینه بالا باشد\n\n"
    "دستورالعمل‌ها:\n"
    "- سؤال‌ها باید واقع‌گرایانه، دقیق و بر اساس داده‌های ارائه‌شده باشند\n"
    "- فقط JSON تولید شود و هیچ متن اضافه‌ای در خروجی نباشد\n"
    "- از ساخت سؤال‌های مبهم یا دارای چند پاسخ صحیح خودداری کنید\n"
    "- هر سؤال فقط باید به یک بیماری مربوط باشد\n"
    "- در متن زیر، هر بیماری با خط جداکننده `--- بیماری ---` آغاز می‌شود و اطلاعات مربوط به همان بیماری بعد از آن آمده است
    + example +
    "اکنون با استفاده از متن زیر که شامل اطلاعات بیماری‌های مختلف است، دقیقاً 50 نمونه مشابه تولید کن:\n\n"
    + disease_text
)
```

```
{
  "question": "کدام یک از علائم زیر برای آب سیاه حاد چشم (گلوکوم حاد) شایع است؟",
  "options": [
    "تاری دید",
    "درد شدید چشم و سردرد",
    "خشکی چشم",
    "التهاب ملتحمه"
  ],
  "answer": "درد شدید چشم و سردرد"
},
{
  "question": "چه عاملی خطر ابتلا به آب سیاه مزمن زاویه باز را افزایش می‌دهد؟",
  "options": [
    "سن بالای 60 سال",
    "مصرف ویتامین A",
    "ورزش منظم",
    "رژیم غذایی کم چرب"
  ],
  "answer": "سن بالای 60 سال"
},
```

همانطور که دیده می‌شود، نتایج به نحوی نیست که می‌خواهیم، یعنی گزینه ها به جای صرفا نام بیماری‌ها، توضیحات هم هستند.

برای همین در ادامه از پرامپت زیر استفاده کردیم:

```
prompt = (
    "شما یک مدل تولید داده تستی برای ارزیابی مدل‌های هوش مصنوعی هستید.\n"
    "وظیفه شما تولید 100 سؤال چندگزینه‌ای (MCQ) بر اساس متن داده‌شده است.\n"
    "هر نمونه باید شامل موارد زیر باشد:\n"
    "- \"question\": یک پرسش بالینی شامل توصیف بیمار، علائم، یا وضعیت مراجعه (نه نام بیماری)\n"
    "- \"options\": چهار نام بیماری مختلف به زبان فارسی (فقط یک بیماری مرتبط با توصیف بیمار باشد)\n"
    "- \"answer\": نام درست بیماری از بین گزینه‌ها\n\n"
    "دستورالعمل‌ها:\n"
    "فقط یکی از گزینه‌ها باید جواب صحیح باشد\n"
    "صورت سؤال نباید شامل نام بیماری باشد، فقط توصیف مرتبط با یک بیماری باشد\n"
    "گزینه‌ها باید فقط نام بیماری ها (کوتاه، دقیق، به زبان فارسی) باشند\n"
    "نباید تولید شود JSON هیچ متن اضافه‌ای خارج از\n"
    "توجه: در متن زیر، هر بیماری با خط جداکننده `--- بیماری ---` شروع می‌شود\n\n"
    + example +
    "اکنون با استفاده از متن زیر که شامل اطلاعات بیماری‌های مختلف است، دقیقاً 100 نمونه مشابه تولید کن:\n\n"
    + disease_text
)
```

```
{
  "question": "بیماری با علائم استفراغ و تب مراجعه کرده است. محتمل‌ترین تشخیص چیست؟",
  "options": [
    "آبسه ریه",
    "آپاندیسیت",
    "آترواسکلروز",
    "آب سیاه"
  ],
  "answer": "آپاندیسیت"
},
{
  "question": "بیماری با علائم قرمزی و سفتی پستان مراجعه کرده است. محتمل‌ترین تشخیص چیست؟",
  "options": [
    "آبسه پستان",
    "آبله مرغان",
    "آترواسکلروز",
    "آسم"
  ],
  "answer": "آبسه پستان"
},
```


که همانطور که دیده می‌شود، نتایج آن همانطور است که انتظار داشتیم.

از کد زیر هم استفاده کردیم:

```python
data = {
    "model": "deepseek/deepseek-chat",
    "messages": [
        {
            "role": "user",
            "content": prompt
        }
    ]
}

# Send request and print the response
response = requests.post(url, headers=headers, json=data)

if response.status_code == 200:
    result = response.json()
    output = result["choices"][0]["message"]["content"]

    try:
        # Save the raw model output to a text file (no parsing, no printing)
        with open("model_test_output_100.txt", "w", encoding="utf-8") as f:
            f.write(output)

        print("Output saved to model_output.txt")

    except Exception as e:
        print("Error saving the output:", e)
else:
    # Report HTTP error from the API response
    print("Error fetching response:", response.status_code, response.text)
```

❖ *ورودی مدل و ارزیابی داده‌ها:*

برای ارزیابی عملکرد مدل آموزش‌دیده، یک اسکریپت ارزیابی طراحی شد که از داده‌های تستی ساختارمند شامل سوال، چهار گزینه و پاسخ صحیح استفاده می‌کند. ساختار سوالات به گونه‌ای است که به مدل یک وضعیت بالینی (clinical scenario) داده می‌شود و مدل باید با توجه به علائم داده‌شده، بیماری مرتبط را تشخیص دهد.

```
import difflib
import csv

alpaca_test_prompt = """### Instruction:
بر اساس علائم و شکایات زیر، تشخیص بیماری را بده.

### Input:
{}

### Response:
"""

correct = 0
total = len(test_data)
results = []

for sample in test_data:
    question = sample["question"]
    options = sample["options"]
    answer = sample["answer"]

    prompt = alpaca_test_prompt.format(question)

    inputs = tokenizer([prompt], return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=64, use_cache=True)
    raw_output = tokenizer.decode(outputs[0], skip_special_tokens=True).strip()
```

```
    if "تشخیص داده شده است" in raw_output:
        raw_output = raw_output.split("تشخیص داده شده است")[0]
    if "(" in raw_output:
        raw_output = raw_output.split("(")[0]
    if ":" in raw_output:
        raw_output = raw_output.split(":")[-1]
    cleaned_output = raw_output.strip()

    closest_matches = difflib.get_close_matches(cleaned_output, options, n=1, cutoff=0.3)
    predicted_answer = closest_matches[0] if closest_matches else "هیچ‌کدام"

    if predicted_answer == answer:
        correct += 1

    results.append({
        "سوال": question,
        "گزینه‌ها": " | ".join(options),
        "پاسخ صحیح": answer,
        "پیش‌بینی مدل": predicted_answer,
        "خروجی خام مدل": cleaned_output
    })

accuracy = correct / total
print(f"دقت نهایی مدل: {accuracy:.2%}")
```

```
with open("model_predictions.csv", "w", newline='', encoding="utf-8-sig") as f:
    writer = csv.DictWriter(f, fieldnames=["سوال", "گزینه‌ها", "پاسخ صحیح", "پیش‌بینی مدل", "خروجی خام مدل"])
    writer.writeheader()
    writer.writerows(results)

print("ذخیره شد 'model_predictions.csv' با تمام نتایج در فایل CSV.")
```

مراحل اجرای کد ارزیابی:

۱. **ساخت قالب پرامپت (Prompt Template):**
مشابه با فرمت آموزش مدل، برای هر نمونه تستی یک `prompt` شامل یک دستور (Instruction) و ورودی (Input) ساخته می‌شود.

۲. **تولید پاسخ توسط مدل:**
با استفاده از توکنایزر و مدل (مبتنی برAlpaca)، پاسخ مدل برای هر `prompt` تولید می‌شود.

۳. **پاک‌سازی پاسخ مدل:**
خروجی خام مدل ممکن است شامل توضیحات اضافی یا ساختار ناهماهنگ باشد. بنابراین چند مرحله ساده پاک‌سازی اعمال می‌شود تا فقط نام بیماری استخراج شود:
   - حذف عبارت‌هایی مانند "تشخیص داده شده است"
   - حذف بخش‌های اضافی داخل پرانتز یا بعد از دو نقطه

۴. **نزدیک‌ترین گزینه به پاسخ مدل (Matching):**
برای انتخاب گزینه‌ای از میان ۴ گزینه موجود، از الگوریتم مقایسه متنی `difflib.get_close_matches` استفاده می‌شود. این روش، شباهت‌های متنی بین پاسخ مدل و گزینه‌ها را بررسی کرده و نزدیک‌ترین تطابق را به عنوان **پاسخ مدل** در نظر می‌گیرد.

۵. **مقایسه پاسخ پیش‌بینی‌شده با پاسخ صحیح:**
اگر پاسخ پیش‌بینی‌شده با `answer` مطابقت داشت، یک پاسخ صحیح شمرده می‌شود. این اطلاعات در لیست نتایج ذخیره می‌گردد.

۶. **محاسبه دقت (Accuracy):**
تعداد پاسخ‌های درست تقسیم بر کل نمونه‌ها، دقت مدل را نشان می‌دهد.

۷. **ذخیره خروجی‌ها:**
تمام نتایج شامل سؤال، گزینه‌ها، پاسخ صحیح، پیش‌بینی مدل، و خروجی خام مدل در فایلCSV ، `model_predictions.csv` ذخیره می‌شود.

خروجی نهایی:

خروجی خام مدل	پیش‌بینی مدل	پاسخ صحیح	گزینه‌ها	سوال
آبسه مغزی	آبسه ریه	آپاندیسیت	آبسه ریه | آپاندیسیت | آترواسکلروز | آب سیاه چشم	بیماری با علائم استفراغ و تب مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آکنه پستان	آبسه پستان	آبسه پستان	آبسه پستان | آبله مرغان | آترواسکلروز | آسم	بیماری با علائم قرمزی و سفتی پستان مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آبله مرغان	آبله مرغان	آبله مرغان	آب سیاه چشم | آبله مرغان | آسم | آبسه ریه	بیماری با علائم خارش و تب مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آب سیاه چشم، نوع مزمن	آبله مرغان	آب مروارید	آپاندیسیت | آبله مرغان | آب مروارید | آسم	بیماری با علائم تاری دید و دید دوگانه مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آب سیاه چشم	آب سیاه چشم	آب سیاه چشم	آبسه ریه | آسم | آب سیاه چشم | آب مروارید	بیماری با علائم حالت تهوع و درد چشم مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آترواسکلروز	آترواسکلروز	آترواسکلروز	آترواسکلروز | آبسه ریه | آب مروارید | آبسه پستان	بیماری با علائم درد قفسه سینه و کاهش خون‌رسانی مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آنفولانزا	آبله مرغان	آبسه ریه	آب مروارید | آب سیاه چشم | آبسه ریه | آبله مرغان	بیماری با علائم تب و تعریق مراجعه کرده است. محتمل‌ترین تشخیص چیست؟
آبسه ریه	آبسه ریه	آبسه ریه	آسم | آترواسکلروز | آبسه ریه | آبله مرغان	بیماری با علائم تب و سرفه خلط‌دار مراجعه کرده است. محتمل‌ترین تشخیص چیست؟

```
دقت نهایی مدل: 74.00%
ذخیره شد 'model_predictions.csv' با تمام نتایج در فایل CSV.
```

- **متریک اصلی:** دقت (Accuracy) مدل روی داده‌های تست چندگزینه‌ای
- **خروجی ساختاریافته:** فایل CSV قابل تحلیل برای بررسی عملکرد مدل به تفکیک سوال
- **روش ارزیابی:** مبتنی بر تطابق معنایی-متنی بین خروجی مدل و گزینه‌ها